CSE331 - Assignment #1

Hojun Lee (20211277)
UNIST
South Korea
hojunlee@unist.ac.kr

1 PROBLEM STATEMENT

In common case, Sorting algorithms primarily prioritize time complexity. Additionally, space complexity and stability can be considered as important evaluation criteria. However, depending on the design approach, the datasets and sizes on which an algorithm performs well may vary. In this paper, we will evaluate algorithm performance on sorted datasets, reversed datasets, random datasets, and partially sorted datasets.

2 BASIC SORTING ALGORITHMS

2.1 Bubble Sort

2.1.1 *Design Rationale.*

Bubble Sort iterates through an array, comparing adjacent elements. If they are not in the correct order, their positions are swapped. Each pass increases the size of the sorted portion of the array by one. The design principle of Bubble Sort is simple and intuitive.

2.1.2 *Time Complexity Analysis.*

The time complexity of bubble sort always involves $n + (n-1) + (n-2) + \cdots 2 + 1 = O(n^2)$ comparisons. Thus, it always has a time complexity of $O(n^2)$. The worst-case scenario occurs when the array is sorted in reverse order because swaps happen at every comparison.

2.1.3 *Space Complexity Analysis.*

Since the only additional memory required is for the variable(s) needed for the swap operation, the space complexity is O(1). Therefore, it is an in-place algorithm.

2.1.4 *Stability.*

Bubble sort is a stable algorithm because when traversing sequentially, elements of equal value maintain their relative positions since they don't get swapped with each other.

2.1.5 *Optimized Bubble Sort with Swap Flag.*

Even with an already sorted array, the basic bubble sort still takes $O(n^2)$ time. By using a **swap_flag**, we can eliminate unnecessary comparisons from the basic bubble sort algorithm: During each pass through the array, the **swap_flag** records whether any swap has occurred If no swap occurred during a complete pass, it means the array is already sorted, and the algorithm can terminate If a swap did occur, sorting continues

With this optimization, it has a best-case time complexity of $O(n)$ for sorted input. However, the worst-case time complexity is still $O(n^2)$.

In experiments, I used this optimized bubble sort due to time limitations.

2.2 Selection Sort

2.2.1 *Design Rationale.*

Selection Sort uses an incremental approach. It maintains a sorted subarray at the beginning of the array, followed by an unsorted subarray. The algorithm sorts by repeatedly finding the minimum element from the unsorted subarray and swapping it with the element at the beginning of the unsorted subarray (which is equivalent to placing it at the end of the sorted subarray), thereby growing the sorted subarray's size. Selection Sort is an intuitive and simple algorithm.

2.2.2 *Time Complexity Analysis.*

The size of the sorted array is increased one element at a time. Therefore, the time complexity is $n+n-1+n-2\cdots+2+1 = \theta(n^2)$.

2.2.3 *Space Complexity Analysis.*

The algorithm operates within the given array, requiring only constant additional memory for temporary variables used in swapping and comparison operations. thus Space Complexity is O(1) and selection sort is in-place algorithm.

2.2.4 *Stability.*

Selection Sort is not a stable sorting algorithm. The sorting proceeds by swapping the first element of the unsorted portion with the element holding the minimum value. It's possible that an element with the same value as the first element exists between the elements being swapped. This swap can alter the relative order of equal elements.

2.3 Insertion Sort

2.3.1 *Design Rationale.*

Insertion Sort uses an incremental approach. It divides the array into a sorted portion at the front and an unsorted portion at the back. The algorithm sorts by taking the first element from the unsorted portion and inserting it into its correct position within the sorted portion. The key design principles are:

- **Simplicity**: Implementation is straightforward, using an intuitive approach of traversing the array and inserting each element in its appropriate position.
- **Adaptivity**: When portions of the array are already sorted, the number of comparisons is minimized, approaching O(n) performance.
- **Online Processing**: Can process data streams in real-time without needing to load all data at once.

2.3.2 *Time Complexity Analysis.*

- Best Case: O(n) when the array is already sorted, as it requires only one pass through the array.

- Worst Case: O(n²) when the array is sorted in reverse order. In this case, comparisons and exchanges occur $1 + 2 + 3 + \cdots + (n-1) = \frac{n(n-1)}{2}$ times.
- Average Case: O(n²) [8]

2.3.3 *Space Complexity Analysis.*
The algorithm operates within the given array, requiring only constant additional memory for temporary variables used in swapping and comparison operations. Thus, space complexity is $O(1)$ and insertion sort is an in-place algorithm.

2.3.4 *Stability.*
insertion sort is a **stable** algorithm. When comparing and exchanging elements in the sorted array, elements with equal values maintain their original relative order since they are not swapped with each other.

2.3.5 *Online Algorithm Properties.*
Insertion Sort functions as an online algorithm, efficiently processing elements one at a time as they are received, making it suitable for streaming data applications.

2.4 Merge Sort

2.4.1 *Design Rationale.*
Merge Sort employs the divide-and-conquer paradigm. It recursively divides the input array into two halves, sorts the two halves, and then merges them to produce the sorted array.

2.4.2 *Time Complexity Analysis.*
The time complexity is consistently $O(n \log n)$ for best, average, and worst cases due to the balanced division and linear-time merge step. the recurrence of running time is $T(n) = 2T(n/2) + O(n)$. By master theorem, its time complexity is $O(nlogn)$.

2.4.3 *Space Complexity Analysis.*
Requires $O(n)$ auxiliary space for the temporary array used during the merge process. It is not an in-place algorithm.

2.4.4 *Stability.*
Merge Sort is a stable sorting algorithm if the merge operation is implemented carefully to prioritize elements from the left subarray when values are equal.

2.5 Heap Sort

2.5.1 *Design Rationale.*
Heap sort uses an incremental approach. Additionally, it utilizes a data structure called a heap to sort efficiently. Heap Sort divides the array into two partitions: the **heap structure** and the **sorted region**. The heap is located in the left partition while the sorted array is positioned in the right partition. The algorithm operates by repeatedly extracting the maximum element from the heap and adding it to the sorted region.

2.5.2 *Time Complexity Analysis.*
- heapify operation takes $O(logn)$ time.
- Building the heap requires $O(n)$ time.
- Extracting the maximum element and restoring the heap requires $O(\log n)$ operations (heapify).
- This extraction and heapify operation is repeated $n-1$ times.
- Overall time complexity: $O(n)+(n-1)\times O(\log n) = O(n \log n)$. [3]

2.5.3 *Space Complexity Analysis.*
All operations can be performed within the original array. The algorithm requires only additional constant memory for temporary variables during swaps. Thus, Space Complexity is $O(1)$ and heap sort is an in-place algorithm.

2.5.4 *Stability.*
the heap sort is not stable. The heapify operations can change the relative order of elements with equal values. The algorithm does not guarantee that equal elements will be extracted in their original order.

2.6 quick sort last element as pivot

2.6.1 *Design Rationale.*
Quick sort is a divide-and-conquer algorithm that partitions an array around a chosen pivot. the last element is selected as the pivot. The array is rearranged so that all elements less than or equal to the pivot are positioned to its left, and all elements greater than the pivot are placed to its right.

2.6.2 *Time Complexity Analysis.*

The recurrence for the running time of Quick Sort is given by:

$$T(n) = T(i) + T(n-i-1) + \Theta(n),$$

where i is the number of elements in the left partition relative to the chosen pivot.

- **Best Case:** In the best-case scenario, the pivot divides the array into two nearly equal halves. Thus, the recurrence becomes:
$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n).$$
By applying the Master Theorem, the time complexity is:
$$T(n) = O(n \log n).$$
- **Worst Case:** In the worst-case scenario, the pivot results in a highly unbalanced partition (e.g., when the smallest or largest element is always chosen as the pivot). The recurrence is:
$$T(n) = T(n-1) + \Theta(n).$$
Expanding this recurrence gives:
$$T(n) = \Theta\big(n + (n-1) + (n-2) + \cdots + 2 + 1\big) = \Theta(n^2).$$
- **Average Case (Expected Value):** For random input data, the expected running time of Quick Sort is:
$$T(n) = \Theta(n \log n).$$
[3]

2.6.3 *Space Complexity Analysis.*
Quick sort is in-place when partitioning. however, recursion requires additional space for new stack:

- **Worst Case:** $O(n)$ due to the recursion stack.
- **Average Case:** $O(logn)$ recursion depth.

2.6.4 Stability.

Quick sort is not stable. The partitioning process may swap non-adjacent elements, thereby changing the relative order of equal elements.

2.6.5 Randomized Quick sort.

This variation selects the pivot randomly. When the pivot is chosen at random, the expected time complexity is $O(n \log n)$. Statistically, as n increases, the performance approaches the expected value. This implies a recursion depth of $O(\log n)$, and the expected space complexity is $O(\log n)$, so on average, it is an in-place algorithm. However, the worst case still exists probabilistically. In this paper, we use this version. [3]

3 ADVANCED SORTING ALGORITHMS

3.1 Library Sort

3.1.1 Design Rationale.

Library Sort improves upon Insertion Sort by introducing gaps within the array. Elements are inserted into this expanded array with spaces, which reduces the cost of shifting elements during insertion. Also, during insertion, elements are shifted toward the nearest gap, reducing the number of swap operations. Periodically, the array is rebalanced to redistribute the gaps evenly. This algorithm is based on the hypothesis that the input data arrives randomly. [1]

Algorithm 1 Rebalance

```
1:  procedure REBALANCE(A, begin, end)
2:      r ← end
3:      w ← end × 2
4:      while r ≥ begin do
5:          A[w + 1] ← gap
6:          A[w] ← A[r]
7:          r ← r − 1
8:          w ← w − 2
9:      end while
10: end procedure
```

Algorithm 2 Library Sort

```
1:  procedure SORT(A)
2:      n ← length(A)
3:      S ← new array of n gaps
4:      for i ← 1 to ⌊log2(n)⌋ + 1 do
5:          for j ← 2^i to 2^(i+1) do
6:              ins ← BINARYSEARCH(A[j], S, 2^(i−1))
7:              insert A[j] at S[ins]
8:          end for
9:      end for
10: end procedure
```

3.1.2 Time Complexity Analysis.

Library Sort's performance is determined by two primary factors: the cost of rebalancing the array and the cost of shifting elements during insertions.

Rebalancing Cost: Library Sort maintains gaps within its array to reduce the shifting overhead during element insertions. Periodically, the algorithm triggers a rebalancing step to redistribute elements and gaps. At each rebalancing level—indexed by i—approximately 2^i elements are involved. The rebalancing cost can be modeled by the geometric series:

$$O(1 + 2 + 4 + \cdots + 2^i),$$

which sums to:

$$\sum_{k=0}^{i} 2^k = 2^{i+1} - 1.$$

Assuming that 2^i is on the order of the input size n, the overall rebalancing cost amortizes to $O(n)$.

Best-case Scenario: In the best-case, every insertion operation takes advantage of a pre-established gap. This means each insertion occurs in constant time $O(1)$ with minimal shifting overhead. Although the rebalancing step incurs an amortized cost of $O(n)$ over n insertions, the resulting best-case overall time complexity remains $O(n)$.

Worst-case Scenario: In the worst-case scenario, insertions force extensive shifts of elements. For example, if every new element is inserted at one end of the array, the shifting cost builds up according to the series:

$$O(1 + 3 + 5 + \cdots + (2n - 1)).$$

the worst-case time complexity becomes $O(n^2)$.

3.1.3 Space Complexity Analysis.

Requires $O(n)$ auxiliary space to store the array with gaps. Thus, library sort is not an in-place algorithm.

3.1.4 Stability.

The library sort is not a stable algorithm. During the insert operation, elements may be shifted either to the left or pushed to the right, which means that the relative order of identical elements is not guaranteed.

3.2 Comb Sort

3.2.1 Design Rationale.

Comb Sort improves on Bubble Sort by comparing and swapping elements that are far apart, using a gap value that starts large and shrinks over iterations. This helps eliminate small values near the end of the list more quickly than Bubble Sort's adjacent swaps. Eventually, the gap becomes 1, at which point Comb Sort functions like a Bubble Sort. [4]

3.2.2 Time Complexity Analysis.

- **Best Case**: $O(n \log n)$ is achievable with optimal shrink factors, but often cited as closer to $O(n)$ if the list is nearly sorted and the gap reduces quickly to 1. A more common bound might be $O(n \log n)$.
- **Average Case**: $O(\frac{n^2}{2^p})$ [5]
- **Worst Case**: $O(n^2)$.

Algorithm 3 Comb Sort

```
procedure CombSort(A)
    n ← Length(A)
    gap ← n
    shrink ← 1.3
    sorted ← false
    while not sorted do
        gap ← ⌊gap/shrink⌋
        if gap ≤ 1 then
            gap ← 1
            sorted ← true
        else
            sorted ← false
        end if
        i ← 0
        while i + gap < n do
            if A[i] > A[i + gap] then
                Swap(A[i], A[i + gap])
                sorted ← false
            end if
            i ← i + 1
        end while
    end while
end procedure
```

3.2.3 *Space Complexity Analysis.*

Comb Sort requires only $O(1)$ auxiliary space for variables such as loop indices, and a temporary variable for swaps. Therefore, it is an in-place algorithm.

3.2.4 *Stability.*

It is unstable because swaps can occur between elements with large gaps, and there may be elements of the same value between the swapped elements.

3.3 Cocktail Shaker Sort

3.3.1 *Design Rationale.*

Cocktail Shaker Sort is a variation of Bubble Sort. It improves performance slightly by sorting in both directions on each pass. It traverses the list from left to right, moving the largest element to its correct position at the end, and then traverses from right to left, moving the smallest element to its correct position at the beginning. This helps move both small values near the end and large values near the start more effectively than standard Bubble Sort. The range of traversal shrinks from both ends with each full pass. [6]

3.3.2 *Time Complexity Analysis.*

- **Best Case**: $O(n)$. Occurs when the list is already sorted or nearly sorted; it can terminate early if a full pass (both directions) results in no swaps.
- **Average Case, Worst Case**: $O(n^2)$. It simply applies bubble sorting in both directions.

3.3.3 *Space Complexity.*

cocktail shaker sort requires only $O(1)$ auxiliary space for variables such as loop indices, and a temporary variable for swaps. Therefore, it is an in-place algorithm.

Algorithm 4 Cocktail Shaker Sort

```
procedure CocktailShakerSort(A)
    n ← Length(A)
    swapped ← true
    start ← 0
    end ← n − 1
    while swapped do
        swapped ← false
        for i ← start to end − 1 do
            if A[i] > A[i + 1] then
                Swap(A[i], A[i + 1])
                swapped ← true
            end if
        end for
        if not swapped then
            break
        end if
        swapped ← false
        end ← end −1
        for i ← end − 1 downto start do
            if A[i] > A[i + 1] then
                Swap(A[i], A[i + 1])
                swapped ← true
            end if
        end for
        start ← start +1
    end while
end procedure
procedure Swap(x, y)
    temp ← x
    x ← y
    y ← temp
end procedure
function Length(List)
    return the number of items in List
end function
```

3.3.4 *Stability.*

Bubble sort is a stable sorting algorithm. Since cocktail shaker sort applies bubble sort in both directions, cocktail shaker sort is also stable.

3.4 Introsort

3.4.1 *Design Rationale.*

Introsort improves upon two drawbacks of Quicksort. The first is the significant overhead of the divide-and-conquer approach for small inputs. The other is that Quicksort's performance heavily depends on selecting good pivots. It begins with Quicksort but monitors the recursion depth. If the depth exceeds a certain limit (proportional to $\log n$), indicating a potential worst-case scenario for Quicksort, it switches to Heapsort, which guarantees $O(n \log n)$ time. For very small partitions , it typically switches to Insertion Sort because Insertion Sort has low overhead and performs well on small or nearly sorted arrays. [7]

3.4.2 *Time Complexity Analysis.*

Algorithm 5 Intro Sort

```
procedure IntroSort(arr, n, max_depth, threshold, recur)
    if n ≤ 1 then
        return
    end if
    if recur ≥ max_depth then
        Call HeapSort(arr, n)
        return
    end if
    if n ≤ threshold then
        Call InsertionSort(arr, n)
        return
    end if
    Select pivot randomly
    Partition array around pivot
    Recursively sort left and right partitions
end procedure
```

- **Best Case**: $O(n \log n)$. Primarily driven by Quicksort or Heapsort phases. If the input is nearly sorted and the threshold for Insertion Sort is reached quickly, it might approach $O(n)$ in practice for the Insertion Sort part, but the overall complexity remains dominated by the other phases for large n.
- **Average Case**: $O(n \log n)$. Leverages Quicksort's excellent average-case speed.
- **Worst Case**: $O(n \log n)$. The Heapsort fallback prevents Quicksort's $O(n^2)$ worst case by switching algorithms when deep recursion occurs.

3.4.3 *Space Complexity Analysis.*

Requires $O(\log n)$ auxiliary space on average, primarily for the recursion stack depth of Quicksort. In the worst-case path for Quicksort, the stack depth could theoretically reach $O(n)$, but the switch to Heapsort limits the practical auxiliary space usage. The depth limit for switching is typically set based on $\log n$, making the overall auxiliary space complexity $O(\log n)$. Thus, intro Sort is in-place.

3.4.4 *Stability.*

- **Unstable Sorting Algorithm**: Neither Quicksort nor Heapsort, the primary components, are stable sorting algorithms[2]. Insertion sort is stable, but its use on small partitions doesn't make the overall sort stable.

3.5 Tournament Sort

3.5.1 *Design Rationale.*

Tournament Sort is a sorting algorithm that conceptually uses a binary tree structure, similar to a sports tournament bracket, to repeatedly find the minimum element among the remaining elements. It's a variation of Selection Sort and shares similarities with Heapsort. The main idea is to shorten the minimum value selection time of Selection Sort. .An initial "tournament" is played among all n elements to find the overall winner. This winner becomes the first element of the sorted output. To find the next winner, the previous winner is effectively replaced, and the tournament is partially replayed only along the path affected by the replacement to find the new minimum among the remaining elements. This process repeats n times. [2]

Algorithm 6 Tournament Sort

```
1:  procedure TournamentSort(A)
2:      n ← Length(A)
3:      Create auxiliary array T of size roughly 2n
4:      Create output array SortedA of size n
5:      BuildTournamentTree(A, T, n)
6:      for i ← 0 to n − 1 do
7:          SortedA[i] ← T[1]
8:          leaf_index ← FindLeafIndex(T, T[1], n)
9:          UpdateTree(T, leaf_index, ∞)
10:     end for
11:     return SortedA
12: end procedure

13: procedure BuildTournamentTree(A, T, n)
14:     tree_leaves_start_index ← ...
15:     for i ← 0 to n − 1 do
16:         T[tree_leaves_start_index + i] ← A[i]
17:     end for
18:     for j ← tree_leaves_start_index + n to Length(T) − 1 do
19:         T[j] ← ∞
20:     end for
21:     for k ← tree_leaves_start_index − 1 downto 1 do
22:         T[k] ← min(T[2k], T[2k + 1])
23:     end for
24: end procedure

25: procedure UpdateTree(T, index, newValue)
26:     T[index] ← newValue
27:     k ← ⌊index/2⌋
28:     while k ≥ 1 do
29:         T[k] ← min(T[2k], T[2k + 1])
30:         k ← ⌊k/2⌋
31:     end while
32: end procedure
```

3.5.2 *Time Complexity Analysis.*

- **Setup Phase**: Building the initial tournament structure typically takes $O(n)$ time (similar to building a heap).
- **Selection Phase**: Finding each subsequent minimum requires updating the tree and finding the new root, which takes $O(\log n)$ time. This is done n times.
- **Overall Time Complexity**: $O(n)+n\times O(\log n) = O(n \log n)$ for all cases.

3.5.3 *Space Complexity.*

Requires $O(n)$ auxiliary space for the tournament tree structure. Thus, tournament sort is not in-place algorithm.

3.5.4 *Stability.*

the tournament sort is not stable. The updatetree operations can

change the relative order of elements with equal values. The algorithm does not guarantee that equal elements will be extracted in their original order.

3.6 Tim Sort

3.6.1 Design Rationale.
Tim sort is an adaptive hybrid sorting algorithm that first divides the input array into segments called *runs* and then merges these runs using a highly optimized process. [9]

Run Composition and Management:

- **Run Identification:** The algorithm scans the array to detect naturally ordered sequences called runs. If a run is in descending order, it is reversed in-place. Runs shorter than a predefined minimum length (*minrun*) are extended using binary insertion sort until they satisfy the minimum size requirement.
- **Run Balancing:** Identified runs are maintained on a stack so that merging operations ensure that smaller runs are merged first. In particular, the two smallest runs at the top of the stack are merged if their combined size is less than or equal to the next run's size. This descending order of run sizes helps keep the merge operations balanced.

Optimized Merging with Binary Search:

- Before merging two runs, binary search is used to detect the portions at the left and right boundaries that are already in the correct order, so these parts can be excluded from the merge.
- The remaining segments are merged by first copying the smaller run into temporary memory. If the left run is smaller, merging proceeds from the left; if the right run is smaller, it proceeds from the right. This strategy reduces extra memory usage by roughly half compared to standard merge sort.

Galloping Mode:

- During merging, if consecutive elements are selected from one run, the algorithm switches into **galloping mode**. In this mode, it advances exponentially (by powers of two) to quickly skip over several elements at once.
- When the exponential (galloping) step overshoots, binary search is applied within that interval to efficiently locate the precise insertion point. This reduces the number of comparisons during the merge.

3.6.2 Time Complexity Analysis.

- **Best Case**: $O(n)$. Occurs when the input is already sorted or consists of a few long natural runs, minimizing the work needed for extension and merging.
- **Average Case**: $O(n \log n)$. Performs comparably to Merge Sort but often faster in practice due to adaptivity.
- **Worst Case**: $O(n \log n)$. Maintains efficiency even on difficult inputs due to the balanced merge strategy.

3.6.3 Space Complexity Analysis.

- Requires temporary storage during the merge phase. In the worst case, it might need space proportional to $n/2$, leading to $O(n)$ auxiliary space complexity. However, due to optimizations (like merging smaller runs into larger ones), the practical space usage is often much less, especially for partially sorted data. For nearly sorted data, it can approach $O(\log n)$ space related to the stack of pending runs. The reference table lists $O(n)$ as the general bound. Thus, tim sort is not in-place.

Algorithm 7 TimSort

```
procedure TimSort(arr, n)
    minirun ← 32
    runs ← empty list                ▷ element is a pair (start, end)
    i ← 0
    while i < n − 1 do
        runend ← min(i + minirun, n − 1)
        i ←makerun(arr, n, i, runend)
        if runs is not empty then
            start ← (last element of(runs)).second + 1
            runs ← runs ∪ {(start, i)}
        else
            runs ← runs ∪ {(0, i)}
        end if
     cond = true
        while cond do
            cond <- CheckAndMergeRuns(arr, runs)
        end while
        i ← i + 1
    end while
    while |runs| ≠ 1 do
        MergeRuns(arr, runs, |runs| - 2, |runs| - 1)
    end while
end procedure
```

(1) CheckAndMergeRuns checks run balancing, and if it does not satisfy the condition, it merges runs to achieve run balancing. If a merge occurs, it returns true; otherwise, it returns false.
(2) MergeRuns merges two runs using optimized merging techniques with binary search and galloping mode.
(3) MAKERUN makes run and return ending index

3.6.4 Stability.
Tim sort utilize merge sort and insertion sort. These are stable algorithms. Thus, tim sort is also stable.

4 EXPERIMENTAL RESULTS AND ANALYSIS

4.1 Experimental Setup

In this study, several sorting algorithms were evaluated based on their performance characteristics using different dataset types. The terminology used is defined as follows:

- **Sorted Dataset**: An array where elements are arranged in non-decreasing (ascending) order.
- **Reversed Dataset**: An array where elements are arranged in non-increasing (descending) order.
- **Random Dataset**: An array where elements are in a random permutation.

- **Partially Sorted Dataset**: Generated from a fully sorted dataset. A subset of elements, determined by the **mixing proportion** (p), is selected. These $p \times n$ elements are then randomly shuffled using the Fisher-Yates algorithm, while the remaining $(1-p) \times n$ elements stay in their sorted positions relative to each other. A mixing proportion of 0 indicates a completely sorted dataset, and 1 indicates a completely random dataset (relative to the original sorted order).

The original experimental data and the source code implementation for each algorithm are available at here

4.2 Execution Time vs. Input Size with Sorted Dataset

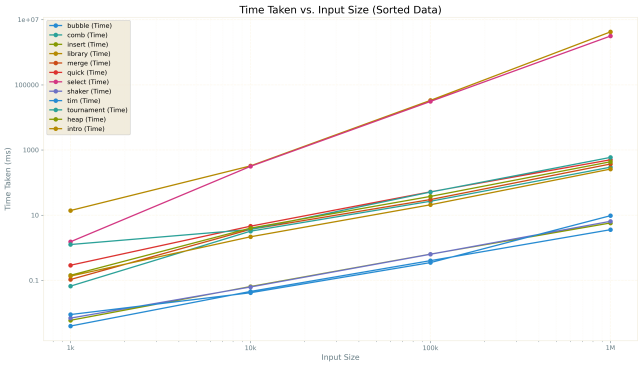

Figure 1: Execution Time vs. Input Size (Sorted Data)

Library Sort and Selection Sort are the slowest, exhibiting O(n²) performance. Meanwhile, Shaker Sort, Tim Sort, Insertion Sort, and Optimized Bubble Sort terminate in O(N) time on sorted data sets, demonstrating excellent performance.

4.3 Execution Time vs. Input Size with Reversed Dataset

Figure 2: Execution Time vs. Input Size (Reversed Data)

Tim sort, due to its adaptability, exhibits the best performance. In contrast, Bubble Sort, Shaker Sort, Selection Sort, Library Sort, and Insertion Sort demonstrate poor performance with an O(n²) time complexity.

4.4 Execution Time vs. Input Size with Random Dataset

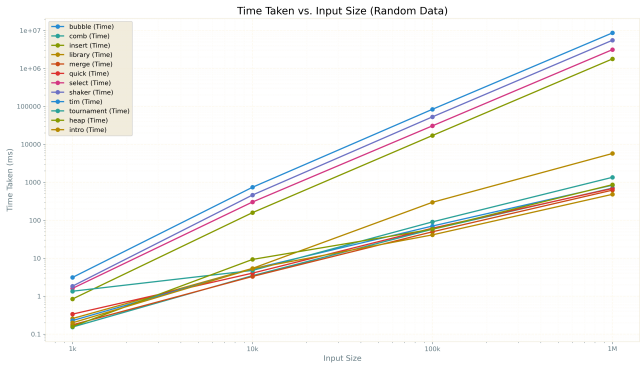

Figure 3: Execution Time vs. Input Size (Random Data)

Bubble Sort, Shaker Sort, Selection Sort, and Insertion Sort exhibit very slow performance, consistent with their $O(n^2)$ average-case time complexity. The remaining algorithms demonstrate good performance.

4.5 Execution Time vs. Mixing Proportion with Partially Sorted Dataset

To investigate performance on partially sorted data, experiments were conducted using datasets of fixed size (1 million elements) with varying mixing proportions. Algorithms with excessively long execution times ($O(n^2)$ group) were tested with larger intervals between mixing proportions. These are referred to as "slow algorithms."

The figure below shows the execution time for the "slow algorithms" (Bubble, Insert, Library, Select, Shaker) versus the mixing proportion.

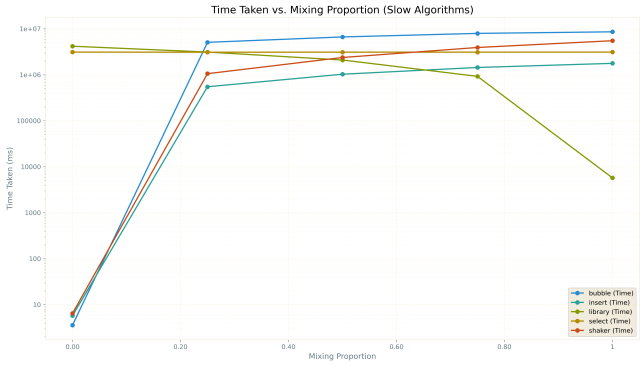

Figure 4: Execution Time vs. Mixing Proportion (Slow Algorithms, N=1M)

Observations for slow algorithms:

- **Adaptive Algorithms**: Shaker Sort, Bubble Sort (optimized), and Insertion Sort demonstrate their adaptive nature. They

perform best (fastest) when the mixing proportion is 0 (fully sorted) and their execution time increases as the proportion of shuffled elements grows, degrading towards their $O(n^2)$ behavior.

- **Library Sort**: Interestingly, Library Sort's performance improves as the mixing proportion increases (i.e., as the data becomes more random). This suggests that its rebalancing strategy and use of gaps are more effective when elements need to be inserted into random locations rather than clustered at one end (as in sorted/reversed cases). Randomness helps elements fall into the prepared gaps more evenly.
- **Selection Sort**: The execution time of Selection Sort shows almost no correlation with the mixing proportion. This confirms its $O(n^2)$ performance is largely independent of the initial order of elements.

The figure below shows the execution time for the remaining ("fast") algorithms versus the mixing proportion.

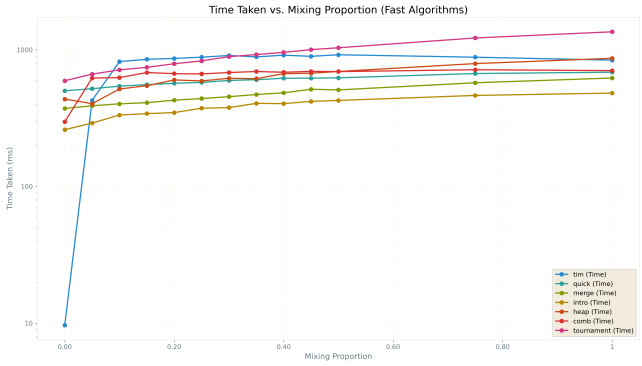

Figure 5: Execution Time vs. Mixing Proportion (Fast Algorithms, N=1M)

Observations for fast algorithms:

- **Tim Sort**: Shows a clear trend of becoming faster as the dataset becomes more sorted (mixing proportion decreases towards 0). This highlights its adaptivity in exploiting existing sorted runs. However, the improvement seems less significant once the mixing proportion is above roughly 0.1, suggesting it handles moderate randomness efficiently but gains substantially from high degrees of sortedness.
- **Comb Sort**: Performance improves significantly as the data becomes fully sorted (mixing proportion 0), approaching its best-case behavior. Its performance degrades as randomness increases.
- **Other $O(n \log n)$ Algorithms**: Most other algorithms in this group (Merge, Quick, Tournament, Heap, Intro Sort) show relatively little variation in execution time with the mixing proportion. Their performance is dominated by the $O(n \log n)$ term, which is less sensitive to partial order than highly adaptive algorithms like Tim Sort or the $O(n^2)$ adaptive sorts. Some minor decreases in runtime as sortedness increases might be attributable to factors like improved cache locality when accessing elements that are already close to their final positions.

4.6 Memory Usage vs. Input Size with Random Data

The figure below presents the peak memory usage (auxiliary space) of the sorting algorithms as a function of input size for random datasets.

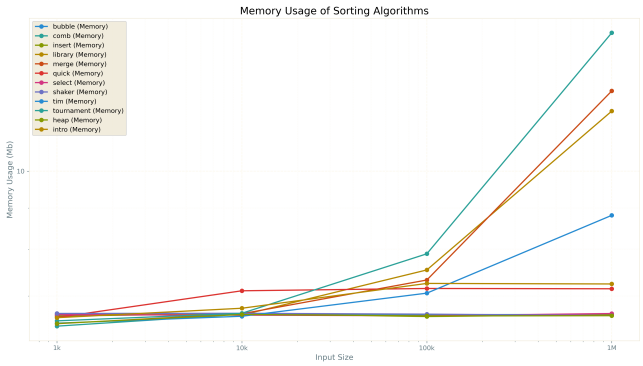

Figure 6: Memory Usage vs. Input Size (Random Data)

Memory usage patterns:

- **In-place Algorithms**: Algorithms designed to be in-place (Bubble, Selection, Insertion, Shaker, Heap, Comb, Intro Sort - noting Intro uses $O(\log n)$ stack) exhibit very low and constant (or near-constant, for stack usage) memory consumption, confirming their $O(1)$ or $O(\log n)$ auxiliary space complexity.
- **Linear Space Algorithms**: Tournament Sort, Merge Sort, and Library Sort form a group with significantly higher memory usage that grows linearly with input size. This is consistent with their theoretical space complexities ($O(n)$ or $\Omega(n)$) due to the need for auxiliary arrays or tree structures proportional to n.
- **Tim Sort**: While having a theoretical worst-case space complexity of $O(n)$, Tim Sort shows memory usage roughly half that of the highest group (Tournament, Merge, Library). This demonstrates the effectiveness of its optimization where it often only needs temporary space for the smaller of the two runs being merged, reducing practical memory footprint compared to a standard Merge Sort implementation.
- **Quick Sort**: Quick Sort's space complexity depends on recursion depth. Its expected space complexity is $O(\log n)$, but the worst-case is $O(n)$ (or even $O(n^2)$ for certain pathological implementations/pivot strategies, though typically $O(n)$ stack depth is the concern). The graph shows Quick Sort having relatively high memory usage for smaller data sizes (e.g., 10k), potentially due to initial stack allocation or overhead. As the data size increases, its memory usage appears to grow more slowly, possibly approaching the expected logarithmic behavior relative to the linear growth of the $O(n)$ algorithms. This suggests the average-case logarithmic behavior dominates for larger inputs.

References

[1] Michael A. Bender, Martin Farach-Colton, and Miguel A. Mosteiro. 2004. Library Sort: An Insertion Sort with O(1) Library Move. In *Proceedings of the European Symposium on Algorithms*. Springer, 136–147.

[2] Prosenjit Bose, Anil Maheshwari, Pat Morin, James Morrison, Michiel Smid, and Jan Vahrenhold. 2006. Fast Algorithms for Median and Selection in Expected Linear Time. *ACM Transactions on Algorithms (TALG)* 2, 2 (2006), 155–174.

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2022. *Introduction to Algorithms* (4th ed.). The MIT Press.

[4] Viliam Geffert, Ján Kollár, and Gabriela Plačková. 2000. Comb Sort: A Forgotten Sorting Algorithm. In *Information Technologies - Applications and Theory (ITAT)*. 29–40.

[5] How.dev. 2023. How to Implement Comb Sort in C++. Online Resource. https://how.dev/answers/how-to-implement-comb-sort-in-cpp Accessed: 2025-04-15.

[6] Janet Incerpi and Robert Sedgewick. 1987. Practical Variations of Shellsort. In *Proceedings of the ACM Symposium on Theory of Computing*. ACM, New York, NY, USA, 35–75.

[7] David R. Musser. 1997. Introspective Sorting and Selection Algorithms. *Software: Practice and Experience* 27, 8 (1997), 983–993.

[8] B. A. Nadel. 1993. Precision complexity analysis: A case study using insertion sort. *Information Sciences* 73, 1-2 (Sept. 1993), 139–189. doi:10.1016/0020-0255(93)90018-H

[9] Tim Peters. 2002. Timsort. https://github.com/python/cpython/blob/main/Objects/listsort.txt.